Конспект: модуль SystemTokenAuth

Модуль реализует **привязку авторизации к конкретному компьютеру**: пользователь может «запомнить этот ПК» и при следующем запуске автоматически войти без ввода логина/пароля, но только на том же компьютере.

---

1. Назначение и контекст

Проблема/Контекст

В приложении ShifRator3 нужна возможность «Войти на этом компьютере» (аналог «Запомнить меня» в браузерах). При этом токен не должен работать на другом ПК — иначе копирование файла дало бы доступ любому.

Решение

Сохраняем в файл **отпечаток системы** (fingerprint) и имя пользователя. При следующем запуске вычисляем отпечаток текущей системы и сравниваем с сохранённым. Совпадение → считаем пользователя авторизованным.

> **Ключевая идея:** отпечаток строится из стабильных данных Windows (серийный номер тома C:\, имя компьютера, имя пользователя), поэтому на другом ПК или под другим пользователем он будет другим.

---

2. Вспомогательные функции (анонимный namespace)

Зачем анонимный namespace?

Код в `namespace { ... }` виден только внутри этого .cpp файла. Функции `WideToNarrow`, `Fnv1aHash`, `ToHex` — внутренняя реализация, не экспортируются в заголовке.

WideToNarrow — преобразование wstring → string (UTF-8)

Windows API возвращает широкие строки (`wchar_t*`). Для записи в файл и для хеша нужна узкая строка в UTF-8.

```
std::string WideToNarrow(const std::wstring& ws)
{
	if (ws.empty()) return {};
	int len = WideCharToMultiByte(CP_UTF8, 0, ws.c_str(), (int)ws.size(), n
	if (len <= 0) return {};
	std::string s(static_cast(len), '\0');
	WideCharToMultiByte(CP_UTF8, 0, ws.c_str(), (int)ws.size(), &s[0], len,
	return s;
}
```

- **Первый вызов** `WideCharToMultiByte` с `nullptr` в качестве буфера — возвращает нужный размер в байтах.
- **Второй вызов** — записывает преобразованные байты в строку `s`. Кодовая страница `CP_UTF8` даёт UTF-8.

Fnv1aHash — детерминированный хеш для отпечатка

Из строки с данными системы нужно получить фиксированный «отпечаток». Используется алгоритм FNV-1a (быстрый, без криптостойкости — для привязки к ПК этого достаточно).

```
uint64_t Fnv1aHash(const std::string& s)
{
	uint64_t h = 14695981039346656037ULL; // FNV offset basis
	for (unsigned char c : s)
		h = (h ^ c) * 1099511628211ULL;
	return h;
}
```

- **14695981039346656037ULL** — стандартная константа FNV-1a для 64 бит.
- **1099511628211ULL** — FNV prime. На каждой итерации: XOR с байтом, затем умножение на prime.
- Одинаковая строка всегда даёт один и тот же `uint64_t`.

ToHex — число в 16-ричную строку

Хеш превращаем в строку из 16 hex-символов для хранения в файле и сравнения.

```
std::string ToHex(uint64_t v)
{
```

```
        std::ostringstream os;
        os << std::hex << std::setfill('0') << std::setw(16) << v;
        return os.str();
}
```

> Итог: отпечаток в файле выглядит как одна строка вида `a1b2c3d4e5f67890` — компактно и однозначно.

---

3. GetSystemFingerprint() — отпечаток текущей системы

Что входит в отпечаток?

Собирается одна строка `raw` из трёх частей, разделённых `|`:

1. **Серийный номер тома C:** — `GetVolumeInformationW(L"C:\\", ...)` возвращает `volSerial`. Меняется при переустановке/смене диска.
2. **Имя компьютера** — `GetComputerNameW`. Меняется при переименовании ПК или в другой сети.
3. **Имя пользователя Windows** — `GetUserNameW`. Разные учётные записи на одном ПК дают разный отпечаток.

Как это работает?

```
std::string raw;
// ... сборка raw = "volSerial|compName|userName"
uint64_t h = Fnv1aHash(raw);
return ToHex(h);  // например "a1b2c3d4e5f67890"
```

Строка `raw` хешируется FNV-1a, результат переводится в hex — это и есть «отпечаток системы».

> Если пользователь переименует компьютер или сменит том, отпечаток изменится и сохранённый токен перестанет подходить. Это ожидаемое поведение «привязки к ПК».

---

4. GetTokenFilePath() — путь к файлу токена

Где хранится файл?

В каталоге приложения внутри **Local AppData** пользователя:

```
%LOCALAPPDATA%\ShifRator3\ShifRator3_system_token.dat
```

Так файл привязан к учётной записи Windows и не доступен другим пользователям этой машины.

Разбор кода

```
if (!SUCCEEDED(SHGetFolderPathW(nullptr, CSIDL_LOCAL_APPDATA, nullptr, 0, buf))
{
        char temp[MAX_PATH];
        if (GetTempPathA(MAX_PATH, temp))
                return std::string(temp) + "ShifRator3\\";
        return "";
}
std::wstring folder = std::wstring(buf) + L"\\ShifRator3\\";
CreateDirectoryW(folder.c_str(), nullptr);
return WideToNarrow(folder) + WideToNarrow(TOKEN_FILENAME);
```

- **SHGetFolderPathW** с `CSIDL_LOCAL_APPDATA` даёт путь к Local AppData.
- Если не удалось (редкий случай) — запасной вариант: папка в `GetTempPath` (временная директория).
- **CreateDirectoryW** создаёт `ShifRator3`, если его ещё нет (игнорирует ошибку «уже существует»).
- Возвращается узкая строка (UTF-8), т.к. `std::ifstream/ofstream` в этом проекте работают с путём в виде `std::string`.

---

5. TryAuthorizeBySystemToken() — проверка «запомненного» входа

Назначение

Вызывается при старте приложения. Проверяет: есть ли файл токена, совпадает ли отпечаток в нём с текущей системой. Если да — возвращает успех и имя пользователя.

Формат файла токена

```
// Две строки, разделитель — TOKEN_SEP ('\n'):
// Строка 1: отпечаток системы (hex)
// Строка 2: username (может быть пустым)
// Пример содержимого файла:
// a1b2c3d4e5f67890
// Ivanov
```

Пошагово

1. Получить путь к файлу; если пусто — вернуть `{false, ""}`.
2. Открыть файл для чтения; если не открылся — `{false, ""}`.
3. Прочитать первую строку → `storedFingerprint`, вторую → `username`.
4. Убрать из обеих строк символы `\r` и `\n` (на случай другой кодировки конца строки).
5. Вычислить `current = GetSystemFingerprint()`.
6. Если `storedFingerprint != current` — токен не для этой системы → `{false, ""}`.
7. Иначе — `{true, username}`.

> Тип возврата `std::pair<bool, std::string>`: первый элемент — успех авторизации по токену, второй — имя пользователя (или пустая строка).

---

6. SaveSystemToken(username) — сохранение «запомнить этот ПК»

Когда вызывать

После успешного входа по логину/паролю, если пользователь выбрал опцию «Запомнить на этом компьютере».

Что делает

```
std::ofstream f(path, std::ios::out | std::ios::trunc);
// ...
f << GetSystemFingerprint() << TOKEN_SEP << username << "\n";
```

- Открывает файл на запись с очисткой (`trunc`).
- Пишет текущий отпечаток системы, разделитель, имя пользователя и перевод строки.

Токен будет валиден только на этом ПК (и для этого пользователя Windows), т.к. отпечаток от них зависит.

7. ClearSystemToken() — выход и удаление токена

Назначение

При явном «Выйти» или отключении «Запомнить этот ПК» нужно удалить файл токена, чтобы при следующем запуске автологин не срабатывал.

```
void ClearSystemToken()
{
        std::string path = GetTokenFilePath();
        if (path.empty()) return;
        DeleteFileA(path.c_str());
}
```

`DeleteFileA` — WinAPI, удаляет файл по пути. Если файла нет, ошибка не критична для логики (токена уже нет).

---

8. Сводка по использованию в приложении

Типичный сценарий

1. **Старт приложения:** вызвать `TryAuthorizeBySystemToken()`. Если вернулось `(true, username)` — считать пользователя авторизованным под `username`, иначе показать форму входа.
2. **Успешный вход по паролю + галочка «Запомнить»:** вызвать `SaveSystemToken(username)`.
3. **Выход / снять «Запомнить»:** вызвать `ClearSystemToken()`.

> **Итог:** модуль SystemTokenAuth реализует привязку авторизации к ПК через отпечаток системы (серийный номер тома, имя компьютера, имя пользователя), хранение отпечатка и имени в файле в AppData и проверку при запуске. Токен не переносится на другой компьютер и не подходит под другой учётной записью Windows.